VIGNAN'S FOUNDATION FOR SCIENCE, TECHNOLOGY AND RESEARCH

Deemed to be University

B.Tech – Artificial Intelligence and Machine Learning

PROJECT REPORT

ONLINE RETAIL SALES DATA PIPELINE

Submitted by

K Manmitha	252FA18020
K Navadeep Reddy	252FA18016

SECTION: AIML-A

Academic Year: 2026

Department of Artificial Intelligence and Machine Learning

ABSTRACT

The Online Retail Sales Data Pipeline is a Python-based data engineering project designed to process, clean, transform, validate, and store online retail transaction data. The project uses an online retail dataset containing transaction-level information such as invoice numbers, product descriptions, quantities, unit prices, customer identifiers, and transaction dates.

The main objective is to develop a reusable and structured Extract, Transform, Load (ETL) pipeline that converts raw retail data into a clean and analysis-ready dataset. The pipeline performs data extraction, preprocessing, feature engineering, validation, checkpointing, and output generation.

Python and Pandas are used for data processing. YAML configuration is used to manage project settings, while logging and exception handling support monitoring and error management. The processed data is exported into CSV and Parquet formats. DVC is used for data version tracking, and Git and GitHub support source-code version control.

The final processed dataset contains 526,052 records and 17 columns, based on the reported pipeline execution results. The project demonstrates how raw business data can be transformed into structured information suitable for further analysis and reporting.

Keywords: ETL, Python, Pandas, Data Cleaning, Feature Engineering, Data Validation, YAML, DVC, CSV, Parquet.

Contents

Abstract **2**

1 INTRODUCTION **4**
1.1 Background 4
1.2 Project Overview 4
1.3 Problem Statement 5
1.4 Objectives 5

2 DATASET DESCRIPTION **5**
2.1 Dataset Overview 5
2.2 Dataset Attributes 5
2.3 Dataset Challenges 6

3 TECHNOLOGIES AND TOOLS USED **6**
3.1 Python 6
3.2 Pandas 6
3.3 NumPy 6
3.4 PyArrow 6
3.5 YAML 7
3.6 Git and GitHub 7
3.7 DVC 7
3.8 Jupyter Notebook 7
3.9 Anaconda 7

4 PROJECT ORGANIZATION **7**
4.1 Folder Responsibilities 8

5 DATA EXTRACTION **8**
5.1 Extraction Process . . . 8
5.2 Importance of Extraction . . . 8

6 DATA CLEANING AND PREPROCESSING **8**
6.1 Introduction . . . 9
6.2 Duplicate Record Handling . . . 9
6.3 Cancelled Invoice Handling . . . 9
6.4 Missing Customer Identifiers . . . 9
6.5 Product Description Handling . . . 9
6.6 Invalid Quantity Handling . . . 9
6.7 Invalid Price Handling . . . 9
6.8 Data Type Consistency . . . 9
6.9 Additional Indicators . . . 9

7 FEATURE ENGINEERING **9**
7.1 Introduction . . . 10
7.2 Total Amount Calculation . . . 10
7.3 Date-Based Features . . . 10
7.4 Customer-Level Features . . . 10
7.5 Benefits of Feature Engineering . . . 10

8 MODULAR PYTHON IMPLEMENTATION **10**
8.1 data_loader.py . . . 10
8.2 data_cleaning.py . . . 11
8.3 feature_engineering.py . . . 11
8.4 validation.py . . . 11
8.5 pipeline.py . . . 11
8.6 Advantages of Modular Design . . . 12

9 YAML CONFIGURATION MANAGEMENT **12**
9.1 Purpose of Configuration . . . 12
9.2 Configuration File . . . 12
9.3 Configuration Benefits . . . 13

10 LOGGING AND EXCEPTION HANDLING **13**
10.1 Logging . . . 13
10.2 Exception Handling . . . 13

11 CHECKPOINTING AND INTERMEDIATE STORAGE **13**
11.1 Purpose of Checkpointing . . . 13
11.2 Checkpoint File . . . 14
11.3 Benefits . . . 14

12 DATA VALIDATION AND QUALITY CHECKS **14**
12.1 Importance of Validation . . . 14
12.2 Validation Checks . . . 14
12.3 Data Quality Report . . . 14
12.4 Importance of the Data Quality Report . . . 14

13 OUTPUT GENERATION AND STORAGE **15**
13.1 CSV Output . . . 15
13.2 Parquet Output . . . 15
13.3 CSV and Parquet Comparison . . . 15
13.4 Output Directory . . . 15

14 VERSION CONTROL AND DATA MANAGEMENT **15**
14.1 Git . . . 15
14.2 GitHub . . . 15
14.3 Data Version Control . . . 16
14.4 Benefits of Version Control . . . 16

15 RESULTS AND OBSERVATIONS **16**
15.1 Pipeline Execution Results . . . 16
15.2 Observations . . . 16
15.3 Project Outcomes . . . 16

16 LIMITATIONS **17**

17 FUTURE ENHANCEMENTS **17**
17.1 Automated Scheduling . . . 17
17.2 Database Integration . . . 17
17.3 Visualization Dashboard . . . 17
17.4 Machine Learning Integration . . . 17
17.5 Cloud-Based Processing . . . 18
17.6 Automated Testing . . . 18

18 CONCLUSION **18**

19 REFERENCES **18**

1 INTRODUCTION

1.1 Background

Online retail platforms generate large volumes of transactional data through customer purchases, product orders, payments, and sales activities. This information is valuable for understanding customer behaviour, identifying sales patterns, and supporting business decisions.

However, raw transaction data is not always suitable for direct analysis. It may contain missing values, duplicate records, invalid quantities, inconsistent descriptions, and other quality issues.

A structured data pipeline is therefore required to process the data systematically and prepare it for analysis.

1.2 Project Overview

The Online Retail Sales Data Pipeline is developed to automate the processing of an online retail transaction dataset. It follows an ETL approach:

- **Extract:** Load the original retail dataset from an Excel file.
- **Transform:** Clean the records, prepare data types, and create useful features.

- **Load:** Save the processed dataset in CSV and Parquet formats.

The implementation is divided into reusable Python modules rather than placing all operations inside one large script.

1.3 Problem Statement

Raw retail transaction data can contain incomplete, duplicated, and invalid information. Processing such data without proper validation may produce incorrect results and unreliable analysis.

The project addresses this problem by creating a reusable ETL pipeline that applies systematic preprocessing, feature engineering, validation, and data storage operations.

1.4 Objectives

The major objectives are:

- To extract data from an Excel-based retail dataset.
- To identify and handle data-quality issues.
- To remove or handle invalid transaction records according to the implemented cleaning logic.
- To create transaction-level and customer-level features.
- To validate the processed dataset.
- To implement reusable Python modules.
- To manage pipeline settings through a YAML configuration file.
- To maintain logs and checkpoints during processing.
- To export processed data into CSV and Parquet formats.
- To track project files and data versions using Git and DVC.

2 DATASET DESCRIPTION

2.1 Dataset Overview

The project uses the Online Retail dataset, which contains transaction records from an online retail business. Each record represents an item associated with an invoice transaction.

The original dataset contains 541,909 records. It includes information about invoices, products, quantities, prices, customers, and transaction dates.

2.2 Dataset Attributes

The important dataset attributes include:

Attribute	Description
InvoiceNo	Unique invoice number associated with a transaction.
StockCode	Product identification code.
Description	Description of the purchased product.
Quantity	Number of units purchased.
InvoiceDate	Date and time of the transaction.
UnitPrice	Price of one unit of the product.
CustomerID	Identifier associated with the customer.
Country	Country associated with the transaction.

Table 1: Important attributes of the original dataset

2.3 Dataset Challenges

The original dataset requires preprocessing before it can be used reliably. The major data-quality challenges considered in the project include:

- Missing customer identifiers.
- Missing or incomplete product descriptions.
- Duplicate transaction records.
- Cancelled invoices.
- Invalid or negative quantities.
- Invalid or negative unit prices.
- Date and numeric data-type consistency.

These issues are handled through the data-cleaning and validation stages.

3 TECHNOLOGIES AND TOOLS USED

3.1 Python

Python is the primary programming language used to implement the pipeline. It provides libraries for data processing, file handling, configuration management, and exception handling.

3.2 Pandas

Pandas is used for loading, manipulating, cleaning, and transforming tabular data. DataFrames provide a convenient structure for performing operations on the retail dataset.

3.3 NumPy

NumPy supports numerical operations and is useful for working with numeric values during data processing.

3.4 PyArrow

PyArrow supports Parquet file operations. Parquet provides a columnar storage format suitable for efficient data storage and analytical workloads.

3.5 YAML

YAML is used to maintain configurable project settings separately from the Python implementation. This makes it easier to modify file paths, output names, preprocessing options, and validation settings.

3.6 Git and GitHub

Git is used for tracking source-code changes. GitHub provides a remote repository for maintaining and sharing the project source code.

3.7 DVC

Data Version Control (DVC) is used to track data-related files and support reproducibility of the data workflow.

3.8 Jupyter Notebook

Jupyter Notebook is used for data exploration and understanding the original dataset before implementing the complete pipeline.

3.9 Anaconda

Anaconda provides the Python environment used during local development and execution.

4 PROJECT ORGANIZATION

The project is organized into separate folders for source code, configuration, data, notebooks, reports, and logs.

```
Online_Retail_Sales_Data_Pipeline/
|
|-- config/
| |-- config.yaml
|
|-- data/
| |-- raw/
| | |-- Online Retail.xlsx
| |-- processed/
| |-- checkpoints/
|
|-- logs/
| |-- pipeline.log
|
|-- notebooks/
| |-- 01_Data_Exploration.ipynb
|
|-- reports/
| |-- data_card.md
| |-- data_quality_report.md
| |-- project_report.tex
|
|-- src/
| |-- data_loader.py
| |-- data_cleaning.py
| |-- feature_engineering.py
```

```
| |-- validation.py
| |-- pipeline.py
|
|-- requirements.txt
|-- README.md
|-- .gitignore
|-- dvc.yaml
```

4.1 Folder Responsibilities

- **config:** Stores YAML-based pipeline settings.
- **data/raw:** Contains the original Excel dataset.
- **data/processed:** Stores the final processed outputs.
- **data/checkpoints:** Stores intermediate processing results.
- **logs:** Stores pipeline execution logs.
- **notebooks:** Contains exploratory data analysis.
- **reports:** Contains project and data-quality documentation.
- **src:** Contains the reusable Python implementation.

This modular structure improves readability, maintainability, and separation of responsibilities.

5 DATA EXTRACTION

5.1 Extraction Process

The extraction stage loads the original Online Retail Excel dataset into a Pandas DataFrame. The input file is maintained inside the raw data directory.

The data loader module is responsible for reading the source file and preparing the dataset for subsequent processing.

The extraction process includes:

1. Accessing the configured raw dataset path.
2. Reading the Excel file.
3. Loading the records into a Pandas DataFrame.
4. Making the extracted data available to the cleaning stage.

5.2 Importance of Extraction

A reliable extraction process ensures that the pipeline consistently reads the intended source file. Keeping the input path configurable makes it easier to reuse the pipeline with another dataset or file location.

The original dataset contains 541,909 records before preprocessing.

6 DATA CLEANING AND PREPROCESSING

6.1 Introduction

Data cleaning is an important stage because raw data may contain errors, incomplete values, and inconsistent records. The project applies preprocessing operations to improve the quality of the transaction dataset.

6.2 Duplicate Record Handling

Duplicate records can affect transaction counts and sales calculations. The cleaning stage identifies and handles duplicate rows to avoid repeated records in the processed dataset.

6.3 Cancelled Invoice Handling

Cancelled invoices are identified and handled as part of the preprocessing process. This prevents cancelled transactions from being treated as ordinary completed sales records.

6.4 Missing Customer Identifiers

Customer identifiers may be missing for certain transactions. The implemented cleaning logic handles missing customer information and maintains the missing-customer indicator where applicable.

6.5 Product Description Handling

Missing product descriptions are handled by replacing unavailable descriptions with the value "Unknown". This provides a consistent representation instead of leaving the description field incomplete.

6.6 Invalid Quantity Handling

Quantity values are checked during preprocessing. Invalid quantity records are handled according to the project's cleaning logic so that unreliable values do not affect subsequent calculations.

6.7 Invalid Price Handling

Unit prices are checked for invalid values. Negative or otherwise invalid price records are handled during preprocessing.

6.8 Data Type Consistency

Date and numeric columns are processed to maintain appropriate data types. Consistent data types are important for date-based analysis and numerical calculations.

6.9 Additional Indicators

The cleaning process includes indicators that preserve useful information about certain data conditions:

- **IsCustomerIDMissing:** Indicates whether the customer identifier is missing.
- **IsZeroPrice:** Indicates whether the unit price is zero.

These indicators help retain information about data conditions for later analysis.

7 FEATURE ENGINEERING

7.1 Introduction

Feature engineering transforms existing data into additional attributes that can provide more useful information for analysis.

The project creates transaction-level and customer-level features from the cleaned retail dataset.

7.2 Total Amount Calculation

The total transaction amount for an item is calculated using quantity and unit price.

$$\text{TotalAmount} = \text{Quantity} \times \text{UnitPrice}$$

This feature represents the total value of the corresponding transaction line.

7.3 Date-Based Features

The invoice date is used to create additional time-related attributes:

- **InvoiceMonth:** Represents the month of the transaction.
- **InvoiceYear:** Represents the year of the transaction.
- **InvoiceDay:** Represents the day of the transaction.

These attributes can support monthly, yearly, and daily sales analysis.

7.4 Customer-Level Features

Customer-related information is used to derive additional features. The project includes:

- **CustomerTotalOrders:** Represents the customer order-count feature.
- **AverageOrderValue:** Represents the average order value associated with a customer.
- **TotalCustomerSpending:** Represents the total spending associated with a customer.

These features can support customer-level analysis and help understand purchasing patterns.

7.5 Benefits of Feature Engineering

Feature engineering provides additional information beyond the original transaction columns. It makes the processed dataset more useful for reporting, customer analysis, and future machine learning applications.

8 MODULAR PYTHON IMPLEMENTATION

The project is implemented using separate Python modules. Each module is responsible for a specific stage of the ETL workflow.

8.1 data_loader.py

This module is responsible for loading the raw Excel dataset. It reads the source file and makes the data available to the pipeline.

Main responsibility:

- Read the source Excel file.

- Load the dataset into a DataFrame.
- Handle errors related to data loading.

8.2 data_cleaning.py

This module performs preprocessing and data-cleaning operations.

Main responsibilities:

- Handle duplicate records.
- Process cancelled invoices.
- Handle missing customer information.
- Process missing product descriptions.
- Handle invalid quantities and prices.
- Prepare cleaned data for feature engineering.

8.3 feature_engineering.py

This module creates additional transaction and customer features.

Main responsibilities:

- Calculate TotalAmount.
- Extract invoice month, year, and day.
- Generate customer-level features.
- Prepare the feature-enhanced dataset.

8.4 validation.py

This module checks the quality of the processed dataset.

The validation stage includes checks for:

- Missing values.
- Duplicate records.
- Invalid quantities.
- Invalid prices.
- TotalAmount calculation consistency.
- Date data-type validity.

8.5 pipeline.py

The pipeline module coordinates the overall execution process. It connects the individual modules and manages the sequence of operations.

The pipeline performs extraction, cleaning, feature engineering, validation, checkpointing, and output generation.

8.6 Advantages of Modular Design

The modular structure provides several advantages:

- Each module has a clearly defined responsibility.
- Individual components can be maintained separately.
- Errors can be identified more easily.
- The code can be reused and extended.
- New processing operations can be added without redesigning the entire project.

9 YAML CONFIGURATION MANAGEMENT

9.1 Purpose of Configuration

The project uses a YAML configuration file to maintain important pipeline settings separately from the source code.

This approach avoids hardcoding multiple file paths and processing options inside the Python modules.

9.2 Configuration File

The following configuration represents the project's settings:

Listing 1: config.yaml

```yaml
paths:
  raw_data: data/raw/Online Retail.xlsx
  checkpoint: data/checkpoints/retail_features_checkpoint.csv
  processed_dir: data/processed
  log_dir: logs

output:
  csv_filename: online_retail_features.csv
  parquet_filename: online_retail_features.parquet

preprocessing:
  remove_cancelled_invoices: true
  remove_missing_customer_ids: true
  remove_duplicates: true
  remove_invalid_quantities: true
  remove_invalid_prices: true

features:
  create_transaction_features: true
  create_customer_features: true

validation:
  check_missing_values: true
  check_duplicates: true
  check_invalid_quantities: true
  check_invalid_prices: true

logging:
```

```
  level: INFO
  filename: pipeline.log
```

9.3 Configuration Benefits

- Centralizes project settings.
- Separates configuration from implementation.
- Makes file paths easier to update.
- Provides control over preprocessing and validation options.
- Improves maintainability and reusability.

The pipeline uses the configuration settings to manage its processing behaviour.

10 LOGGING AND EXCEPTION HANDLING

10.1 Logging

Logging is used to record important events during pipeline execution. It helps monitor the progress of processing and identify where an error occurs.

The project maintains its log file inside the logs directory.

The configured log filename is:

`logs/pipeline.log`

The logging level is configured as INFO. This supports recording general execution information and relevant processing events.

10.2 Exception Handling

Exception handling helps prevent unexpected failures from terminating the application without useful information.

The pipeline uses error-handling mechanisms to manage problems that may occur during operations such as:

- Reading the input dataset.
- Accessing configured paths.
- Performing data transformations.
- Validating processed data.
- Saving output files.

Logging and exception handling improve the reliability and maintainability of the pipeline.

11 CHECKPOINTING AND INTERMEDIATE STORAGE

11.1 Purpose of Checkpointing

Checkpointing is used to save an intermediate processing result. It helps preserve processed data at an important stage of the pipeline.

Instead of depending entirely on the original input for every subsequent operation, an intermediate checkpoint can be used to support recovery and inspection.

11.2 Checkpoint File

The project maintains the checkpoint at:

`data/checkpoints/retail_features_checkpoint.csv`

The checkpoint stores an intermediate feature-enhanced dataset.

11.3 Benefits

- Preserves intermediate processing results.
- Helps inspect data during development.
- Supports recovery-oriented workflows.
- Makes debugging easier.
- Reduces the need to repeat earlier stages when the checkpoint is reusable.

12 DATA VALIDATION AND QUALITY CHECKS

12.1 Importance of Validation

Validation checks whether the processed dataset satisfies important data-quality conditions. It helps identify errors that may remain after preprocessing.

12.2 Validation Checks

The project performs checks on the processed dataset.

Validation	Purpose
Missing-value check	Identifies missing values in the processed data.
Duplicate check	Checks whether duplicate rows remain.
Quantity check	Identifies invalid or negative quantity values.
Price check	Identifies invalid or negative price values.
Amount check	Verifies the TotalAmount calculation.
Date check	Checks the validity of the date data type.

Table 2: Data validation checks

12.3 Data Quality Report

The data quality report documents the results of the validation process. It provides a summary of the processed dataset and the checks performed.

The report is maintained in the reports directory as:

`reports/data_quality_report.md`

12.4 Importance of the Data Quality Report

The report provides evidence that validation was performed after data processing. It also makes the results easier to inspect and communicate.

13 OUTPUT GENERATION AND STORAGE

13.1 CSV Output

The pipeline exports the processed dataset in CSV format.

Output filename:

`online_retail_features.csv`

CSV is a widely supported text-based format. It can be opened using spreadsheet applications and loaded by common data-analysis tools.

13.2 Parquet Output

The processed dataset is also exported in Parquet format.

Output filename:

`online_retail_features.parquet`

Parquet is a columnar storage format. It is useful for analytical workloads and can provide efficient storage and reading of structured datasets.

13.3 CSV and Parquet Comparison

Feature	CSV	Parquet
Format	Text-based	Columnar binary format
Readability	Human-readable	Requires compatible software
Usage	Data exchange and spreadsheets	Analytical processing
Storage	Generally larger	Often more compact

Table 3: Comparison of output formats

13.4 Output Directory

The final output files are stored in the configured processed-data directory:

`data/processed/`

Maintaining both formats provides flexibility for different data-analysis requirements.

14 VERSION CONTROL AND DATA MANAGEMENT

14.1 Git

Git is used to maintain the history of source-code changes. It helps track modifications made during project development and supports organized collaboration.

The project uses Git to manage changes to Python files, configuration files, documentation, and other source-code components.

14.2 GitHub

GitHub is used as the remote repository for the project. It provides a centralized location for maintaining the project source code and documentation.

Project repository:

https://github.com/Manmitha-6/Online_Retail_Sales_Data_Pipeline

14.3 Data Version Control

DVC is used to track data-related files and support data version management.

Data files can be large and may change independently of source code. DVC provides a structured way to manage such files alongside the Git-based project workflow.

14.4 Benefits of Version Control

- Maintains a history of source-code changes.
- Supports project organization.
- Helps track modifications.
- Supports reproducibility of data workflows.
- Makes the project easier to maintain.

15 RESULTS AND OBSERVATIONS

15.1 Pipeline Execution Results

The pipeline processes the original online retail dataset and produces a feature-enhanced output dataset.

Based on the reported execution results, the dataset statistics are:

Metric	Result
Original dataset records	541,909
Processed dataset records	526,052
Processed dataset columns	17
Duplicate rows remaining	0
Negative quantities remaining	0
Negative prices remaining	0
Cancelled invoices remaining	0
TotalAmount calculation errors	0
Invalid date data type	False

Table 4: Reported pipeline execution results

15.2 Observations

The pipeline reduced the original dataset from 541,909 records to 526,052 processed records.

The processed dataset contains 17 columns, including the additional features created during feature engineering.

The reported validation results indicate that no duplicate rows, negative quantities, negative prices, or cancelled invoices remained in the final dataset. The TotalAmount calculation check also reported zero errors.

15.3 Project Outcomes

The project successfully demonstrates the implementation of a structured data-processing workflow with:

- Excel-based data extraction.
- Data cleaning and preprocessing.
- Transaction and customer feature engineering.
- Modular Python implementation.
- YAML-based configuration.
- Logging and exception handling.
- Intermediate checkpoint storage.
- Data validation and quality reporting.
- CSV and Parquet output generation.
- GitHub and DVC-based version management.

16 LIMITATIONS

Although the project implements the major stages of an ETL workflow, there are areas that can be improved.

- The current pipeline is designed around a local dataset and local execution environment.
- Processing is not currently presented as a continuously running real-time system.
- The project focuses on data preparation rather than advanced predictive modelling.
- The pipeline can be extended to support additional input formats and larger datasets.
- Further automated testing can improve confidence in individual processing modules.

These limitations provide opportunities for future development.

17 FUTURE ENHANCEMENTS

The project can be extended in several ways to improve its functionality and practical value.

17.1 Automated Scheduling

The pipeline can be integrated with a scheduling system to run automatically at predefined intervals.

17.2 Database Integration

A database such as PostgreSQL or MySQL can be integrated to store processed retail data and support structured querying.

17.3 Visualization Dashboard

A dashboard can be developed to display sales trends, customer behaviour, product performance, and monthly revenue.

17.4 Machine Learning Integration

Machine learning techniques can be introduced for sales forecasting, customer segmentation, and purchase-pattern analysis.

17.5 Cloud-Based Processing

The pipeline can be extended to cloud storage and processing platforms to support larger datasets and scalable execution.

17.6 Automated Testing

Unit tests and integration tests can be added to verify individual modules and the complete pipeline.

18 CONCLUSION

The Online Retail Sales Data Pipeline demonstrates the development of a structured ETL workflow using Python and data-processing tools.

The project begins with the extraction of raw online retail transaction data from an Excel file. The extracted records are then cleaned and transformed through reusable Python modules. Feature engineering adds transaction-level and customer-level attributes that make the dataset more useful for analysis.

YAML configuration provides centralized control over project settings. Logging and exception handling support monitoring and error management, while checkpointing preserves intermediate processing results. Validation checks help assess the quality of the final dataset.

The pipeline generates processed outputs in CSV and Parquet formats. Git and GitHub are used for source-code version management, while DVC supports data version tracking.

The final reported output contains 526,052 records and 17 columns. The reported validation results demonstrate that the major data-quality checks were satisfied.

Overall, this project provides practical experience in Python programming, data engineering, ETL development, data cleaning, feature engineering, validation, configuration management, and version control. It also establishes a foundation for future work involving data visualization, database integration, and machine learning applications.

19 REFERENCES

1. Pandas Documentation, https://pandas.pydata.org/docs/
2. Python Documentation, https://docs.python.org/3/
3. NumPy Documentation, https://numpy.org/doc/
4. PyArrow Documentation, https://arrow.apache.org/docs/python/
5. DVC Documentation, https://dvc.org/doc
6. Git Documentation, https://git-scm.com/doc
7. GitHub Documentation, https://docs.github.com/
8. YAML Specification, https://yaml.org/spec/
9. Online Retail Dataset, UCI Machine Learning Repository, https://archive.ics.uci.edu/